

Convolutional Neural Networks

- From pixels to features: what a convolution does
- Shapes, padding/stride/dilation
- receptive field
- Pooling
- A Convolutional Neural Network (CNN)
- Interpretability, pitfalls, and practical checklists

Why CNNs?

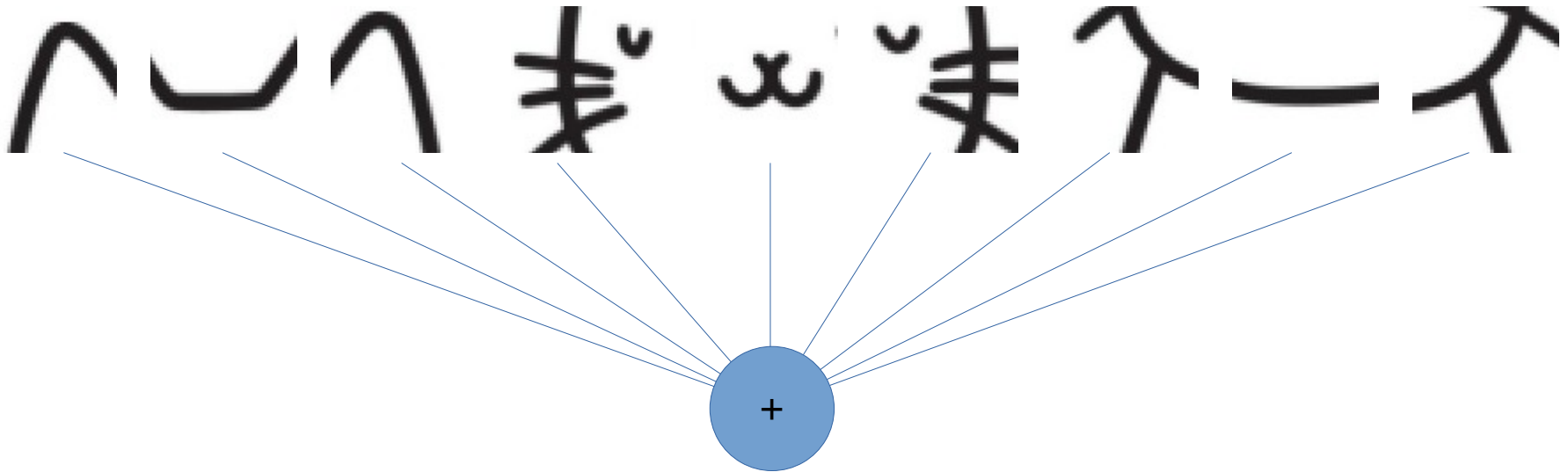
- **Locality: nearby pixels are related**
- Translation equivariance: features shift with objects. If a feature (like an cat) is detected in one part of an image, the filter will detect the same feature in the same way even if the cat is moved to a different position.
- Weight sharing: same filter applied across the image $\Rightarrow$ far fewer params than MLP
- Vision: used in classification, detection, segmentation
- Also used in Audio/spectrograms, time-frequency signals
- Also used with Transformers (ConvNeXt)

Locality: nearby pixels are related



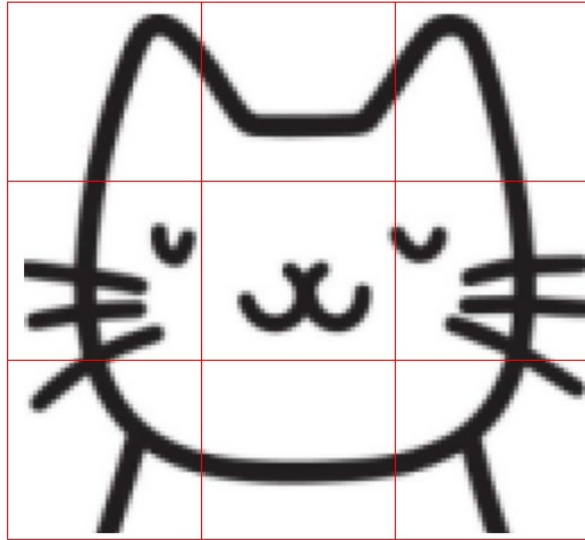
What is this a picture of?

Locality: nearby pixels are related



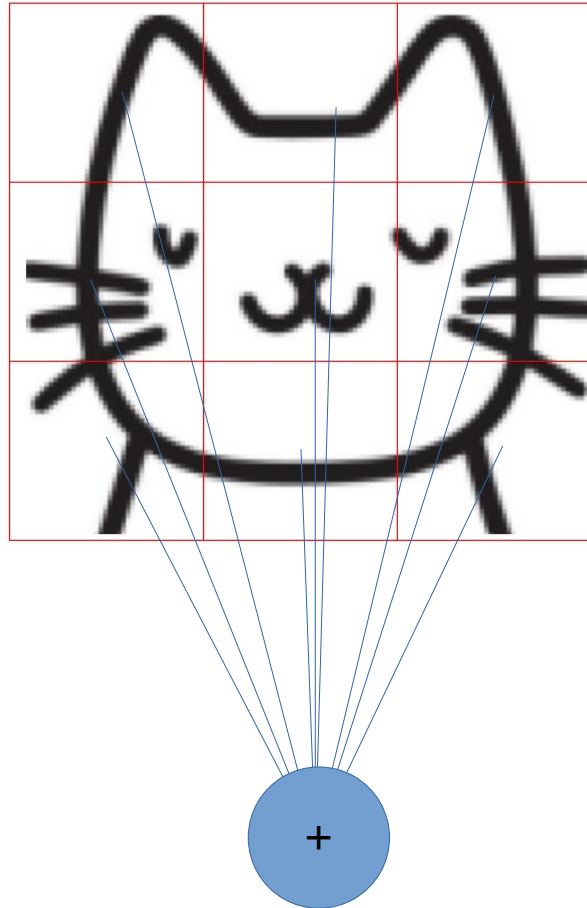
A fully connected Net (MLP) sees a picture this way
It has to figure out how pixels are spatially related

Locality: nearby pixels are related



A conv net sees it this way

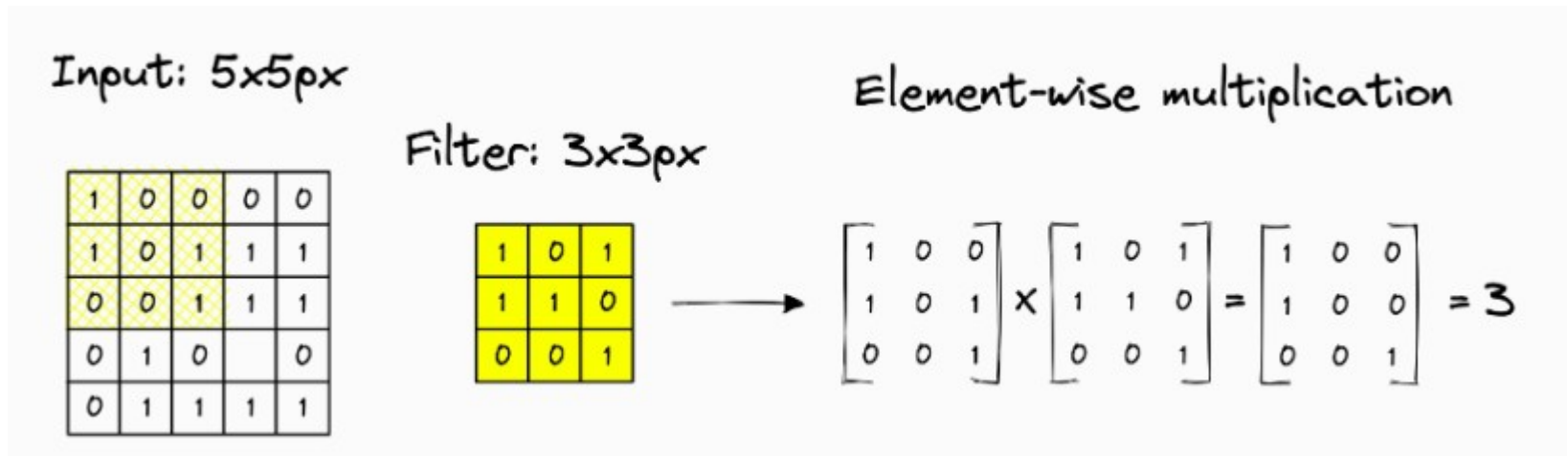
Locality: nearby pixels are related



A conv net sees it this way
Notice that pixels are correctly ordered spatially

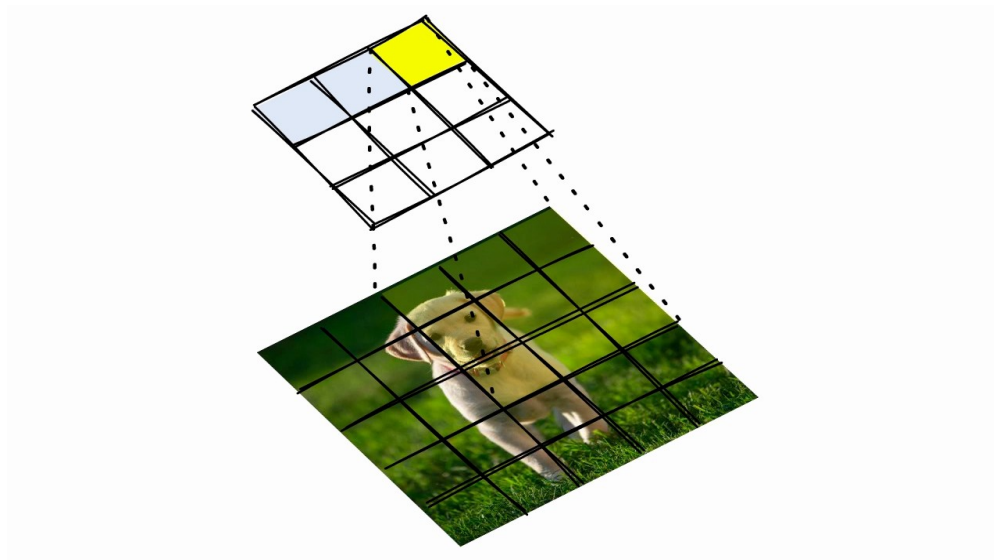
What is a Convolution?

- Learned filter (kernel) slides spatially over input; outputs a feature map
- The following shows a filter applied to upper left of an image. It produces the number 3
- Trained to detect LOCAL features (edges, textures, etc)



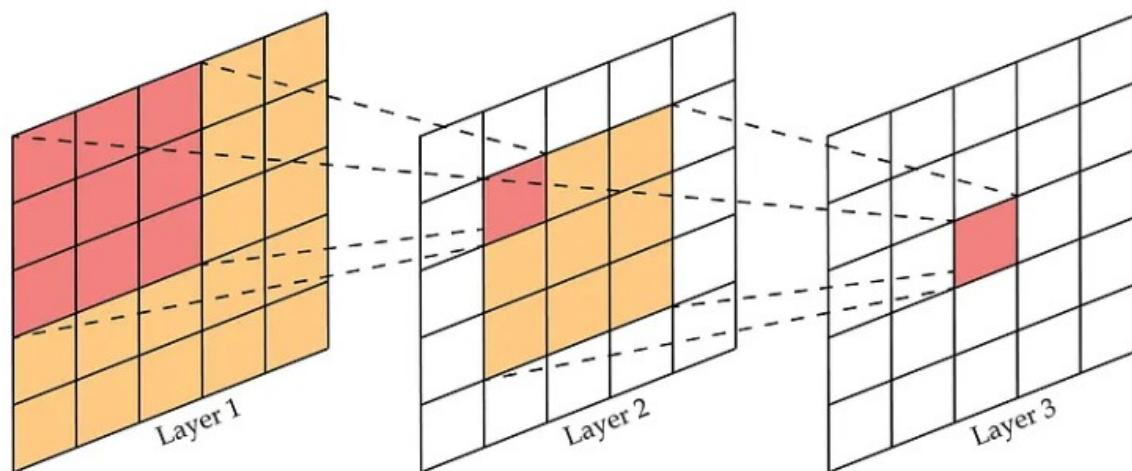
What is a Convolution?

- This filter slides across the entire image, from left to right and top to bottom
- The output is called a feature map



What is a Receptive Field

- Region of input a unit can “see”
- Grows with depth, kernel size, and stride/dilation



The red square in Layer 2 sees the upper 3x3 red squares in Layer 1
But the red square in Layer 3 sees the 9 squares in Layer 2,
each of these was created by the 3x3 filter sliding across Layer 1
The red square in layer 3 effectively sees the entirety of Layer 1

Padding, Stride, Dilation

- Padding: Controls size of output volume by 'picture framing' image with empty pixels
- Stride: downsamples, larger step $\Rightarrow$ smaller feature maps
- Dilation expands receptive field without extra params

Go to this Convolution Visualizer to see how these parameters work together

Calculating Size of Output

For these four hyperparameters:

Filter (Kernel) size F,

Stride S,

Amount of zero padding P

W = width

H = Height

D = Depth

And an input volume of $W1 * H1 * D1$. The receptive field size is:

- $W2 = (W1 - F + 2P) / S + 1$
- $H2 = (H1 - F + 2P) / S + 1$ (i.e. width and height are computed equally by symmetry)

The total number of learnable parameters:

$F * F * D + 1$ bias

Ex:

Image = $28 * 28 * 3$

Filter = $3 * 3$

Padding = 1

stride = 1

$W2 = H2 = (28 - 3 + 2 * 1) / 1 + 1 = 28$

Number trainable parameters:

Conv filter:

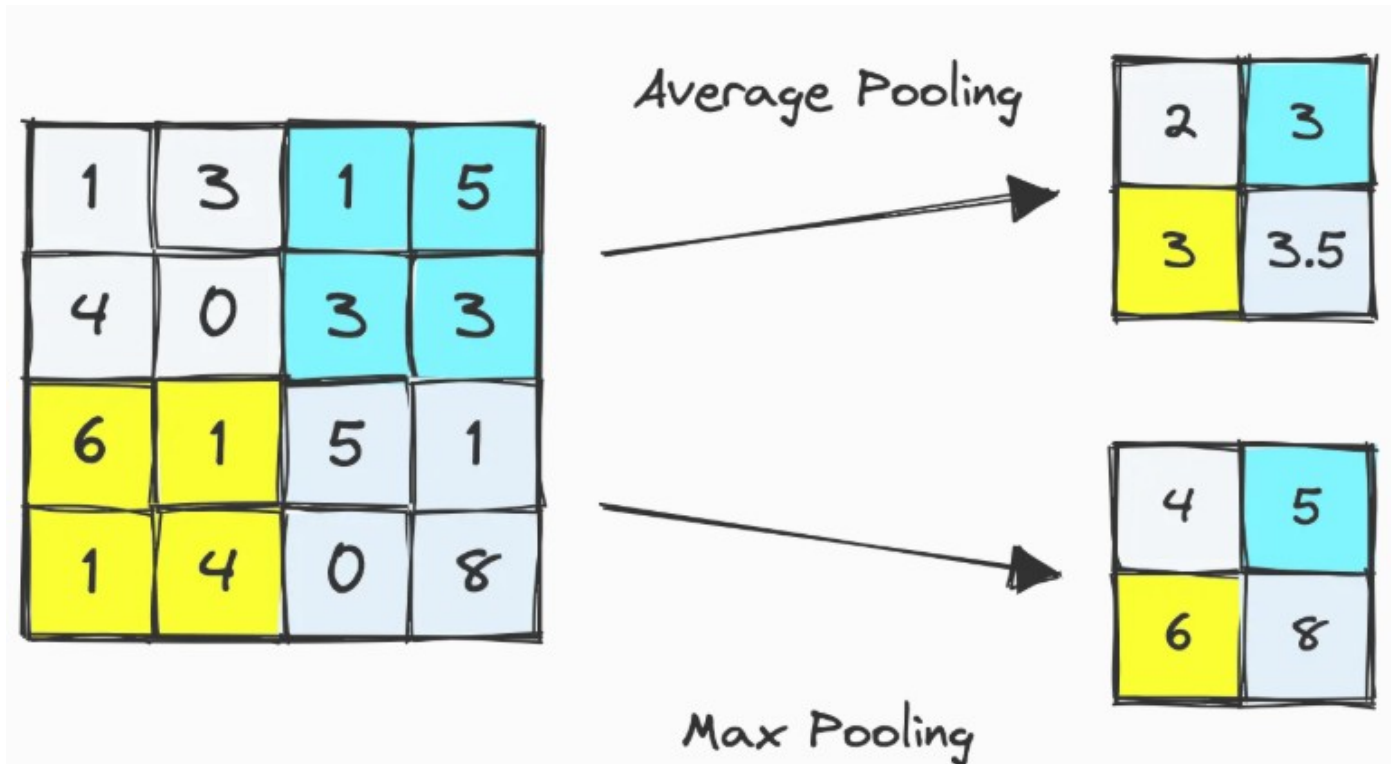
$$3 * 3 * 3 + 1 = 28$$

Fully Connected (MLP):

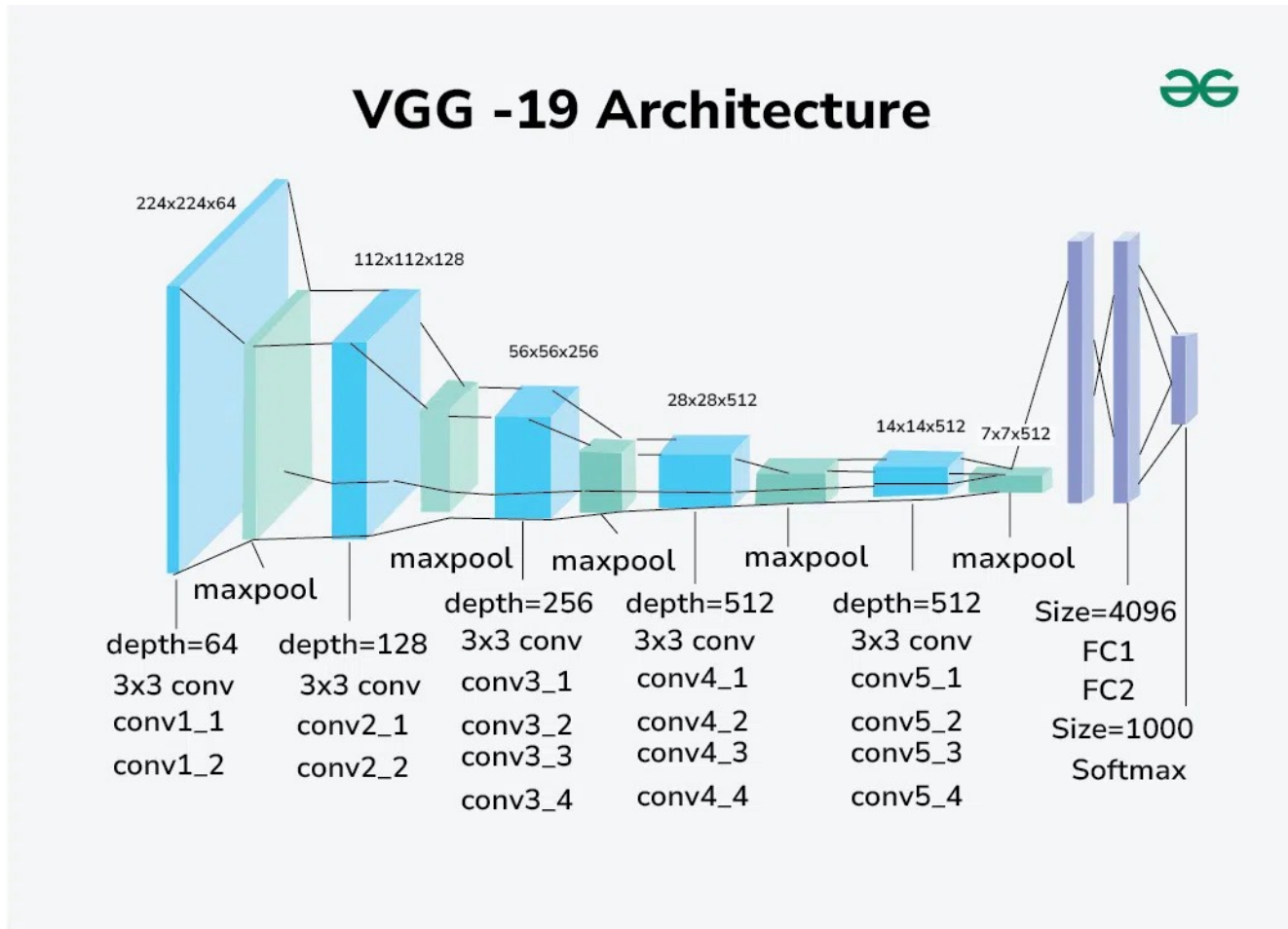
$$28 * 28 * 3 + 1 = 2353$$

Pooling

- A way to downsample a layer (and reduce its size)
- Information across several pixels is compressed into a single activation
- 2 common



A Convolutional Neural Net



Classic → Modern CNN Patterns

- LeNet → AlexNet/VGG → ResNet (skip connections)
- Inception (multi scale), MobileNet/EfficientNet (efficiency)
- ConvNeXt (modern convs, ViT-inspired)

Small CNN in PyTorch

```
import torch, torch.nn as nn
x = torch.randn(8, 3, 64, 64)
model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.BatchNorm2d(16),
    nn.MaxPool2d(2), # 64→32
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2), # 32→16
    nn.Flatten(),
    nn.Linear(32*16*16, 10)
)
print(model(x).shape)
```

Training Details that Matter

- Optimizers: AdamW or SGD+momentum; weight decay
- LR schedules: cosine, OneCycle; AMP; grad clipping
- Normalize inputs; careful splits; strong augmentation

Transfer Learning (coming soon)

```
from torchvision import models
base = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in base.parameters(): p.requires_grad_(False)
base.fc = nn.Linear(base.fc.in_features, num_classes)
# Train head; then unfreeze last block with small LR
```

Evaluation & Interpretability

- Accuracy, confusion matrix, macro-F1
- Grad-CAM / saliency – these let you see what the CNN thought was important
- Failure case review – what model was most confidently wrong about, look at these images to see why.

Common Pitfalls & Debugging

- Shape/order mix-ups (NCHW vs NHWC)
- Overfitting small data (use Data augmentation to generate synthetic data)
- LR too high/low; BatchNorm train/eval modes

Key Takeaways

- Convs = local, shared filters → efficient, fewer parameters than MLP
- Receptive Field /stride/dilation balance detail vs context